ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG CAO ĐẲNG LÝ TỰ TRỌNG THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

XÂY DỰNG HỆ THỐNG QUẢN LÝ NHẬP - XUẤT - TỒN KHO
ỨNG DỤNG LARAVEL FRAMEWORK & RESTFUL API

ĐỒ ÁN CHUYÊN NGÀNH
NGÀNH: LẬP TRÌNH MÁY TÍNH

Họ và tên sinh viên thực hiện:
1. Thái Anh Kiệt MSSV: 25002447
2. Trần Phúc Khang MSSV: 25002446

Khoa: CÔNG NGHỆ THÔNG TIN
Lớp: 25C2-LTM1 Khóa: 2025

Thành phố Hồ Chí Minh, tháng 11 năm 2026

ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG CAO ĐẲNG LÝ TỰ TRỌNG THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

XÂY DỰNG HỆ THỐNG QUẢN LÝ NHẬP - XUẤT - TỒN KHO
ỨNG DỤNG LARAVEL FRAMEWORK & RESTFUL API

ĐỒ ÁN CHUYÊN NGÀNH
NGÀNH: LẬP TRÌNH MÁY TÍNH

Họ và tên sinh viên thực hiện:
1. Thái Anh Kiệt MSSV: 25002447
2. Trần Phúc Khang MSSV: 25002446

Khoa: CÔNG NGHỆ THÔNG TIN
Lớp: 25C2-LTM1 Khóa: 2025

Giảng viên hướng dẫn: Th.S. Mai Chiếm Tuấn

Thành phố Hồ Chí Minh, tháng 11 năm 2026

NHIỆM VỤ ĐỀ TÀI ĐỒ ÁN CHUYÊN NGÀNH

1. Nội dung và các yêu cầu cần giải quyết trong nhiệm vụ đề tài:

a. Nội dung:

- Khảo sát thực trạng bài toán quản lý kho hàng tại các doanh nghiệp vừa và nhỏ: quy trình nhập hàng, xuất bán, kiểm đếm tồn kho và phát hiện các hạn chế của phương pháp thủ công.
- Nghiên cứu kiến trúc phân tầng (Multi-tier Architecture) với mô hình MVC kết hợp Service Layer Pattern trong Laravel Framework.
- Nghiên cứu và hiện thực giải pháp kiểm soát giao dịch đồng thời (Concurrency Control) áp dụng Database Transaction kết hợp Pessimistic Locking (SELECT ... FOR UPDATE) để chống lỗi Race Condition và âm kho.
- Xây dựng hệ thống xác thực RESTful API theo chuẩn Stateless sử dụng Laravel Sanctum Bearer Token.
- Hiện thực mô hình phân quyền người dùng theo vai trò (RBAC - Role-Based Access Control) cho hai nhóm đối tượng: Quản trị viên (Admin) và Nhân viên thủ kho (Warehouse Staff).
- Xây dựng hệ thống nhật ký kiểm toán biến động kho tự động (Inventory Audit Log) truy vết 100% lịch sử giao dịch.
- Thiết kế giao diện Web hiện đại, xây dựng Bảng điều khiển (Dashboard) với biểu đồ trực quan hóa dữ liệu qua Chart.js và tính năng tự động cảnh báo tồn kho dưới ngưỡng an toàn (min_stock).
- Thực hiện kiểm thử tự động toàn diện với PHPUnit (Feature Tests) và kiểm thử giao tiếp API với Postman Collection.

b. Các yêu cầu cần giải quyết:

- Tính toàn vẹn dữ liệu: Đảm bảo số lượng tồn kho luôn chính xác khi nhiều nhân viên cùng thực hiện xuất hoặc nhập hàng đồng thời; ngăn chặn triệt để trường hợp hàng tồn kho bị âm.
- Tính bảo mật và an toàn: Mật khẩu người dùng được băm an toàn bằng thuật toán Bcrypt; các Endpoint API được bảo vệ nghiêm ngặt bằng Bearer Token; phân quyền tác vụ chuẩn xác qua Middleware.
- Tính trực quan và tiện ích: Hỗ trợ tìm kiếm, lọc nhanh theo danh mục/nhà cung cấp; in ấn phiếu nhập kho và phiếu xuất kho chuẩn khổ giấy A4; cảnh báo trực quan các mặt hàng sắp hết.
- Tính mở rộng và tích hợp: Cung cấp 28 Endpoint RESTful API chuẩn mực, định dạng JSON nhất quán, sẵn sàng kết nối với ứng dụng di động hoặc máy quét mã vạch trong tương lai.

- Môi trường triển khai: Hoạt động ổn định trên nền tảng máy chủ XAMPP (Apache, MySQL/MariaDB) và PHP 8.3.

2. Các tài liệu, số liệu cần thiết:

- Phiếu đăng ký đề tài đồ án chuyên ngành đã được Khoa Công nghệ Thông tin phê duyệt.
- Tài liệu quy chuẩn nghiệp vụ quản lý kho vận, mẫu biểu phiếu nhập kho, phiếu xuất kho chuẩn theo quy định kế toán.
- Tài liệu kỹ thuật chính thức của Laravel Framework (v11/12), Laravel Sanctum và PHP 8.3.
- Tài liệu hướng dẫn giao thức RESTful API Specification và thư viện biểu đồ Chart.js.
- Các công trình nghiên cứu và bài báo khoa học quốc tế về kỹ thuật Concurrency Control và cơ chế khóa bản ghi trong CSDL quan hệ.

3. Địa điểm thực hiện đồ án chuyên ngành:

Trường Cao đẳng Lý Tự Trọng Thành Phố Hồ Chí Minh (Số 390 Hoàng Văn Thụ, Phường 4, Quận Tân Bình, TP. Hồ Chí Minh).

ĐÃ NHẬN NHIỆM VỤ ĐTĐACN	**ĐÃ GIAO NHIỆM VỤ ĐTĐACN**
Sinh viên thực hiện	**Giảng viên hướng dẫn**
(Ký và ghi rõ họ tên)	**(Ký và ghi rõ họ tên)**
Thái Anh Kiệt	**Trần Phúc Khang**
	Th.S. Mai Chiếm Tuấn

GIẢNG VIÊN HƯỚNG DẪN ĐỀ TÀI ĐỒ ÁN CHUYÊN NGÀNH

Họ và tên giảng viên hướng dẫn: Mai Chiếm Tuấn
Học hàm, học vị: Thạc sĩ
Đơn vị công tác: Khoa Công nghệ Thông tin – Trường Cao đẳng Lý Tự Trọng TP.HCM

Nội dung hướng dẫn:
1. Định hướng phương pháp tiếp cận bài toán, phân tích yêu cầu nghiệp vụ quản lý kho hàng và hướng dẫn lập kế hoạch chi tiết cho 15 tuần thực hiện đồ án.
2. Hướng dẫn thiết kế kiến trúc hệ thống đa tầng (MVC + Service Layer Pattern) trên nền tảng Laravel Framework và chuẩn RESTful API.
3. Hướng dẫn chuyên sâu về kỹ thuật kiểm soát giao dịch đồng thời (Concurrency Control), nguyên lý ACID và ứng dụng Pessimistic Locking (lockForUpdate) trong Database MySQL.
4. Định hướng triển khai cơ chế xác thực bảo mật API bằng Laravel Sanctum và phân quyền người dùng RBAC.
5. Hướng dẫn xây dựng kịch bản kiểm thử tự động với PHPUnit (Feature Tests), kiểm thử API bằng Postman và hoàn thiện báo cáo khoa học theo quy chuẩn của Nhà trường.

Đề tài đồ án chuyên ngành được giao ngày 05 tháng 09 năm 2026.
Yêu cầu phải hoàn thành xong trước ngày 30 tháng 11 năm 2026.

ĐÃ NHẬN NHIỆM VỤ ĐTĐACN	ĐÃ GIAO NHIỆM VỤ ĐTĐACN
Sinh viên	**Giảng viên hướng dẫn**
(Ký và ghi rõ họ tên)	**(Ký và ghi rõ họ tên)**
Thái Anh Kiệt Trần Phúc Khang	**Th.S. Mai Chiếm Tuấn**

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

PHIẾU NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN ĐỒ ÁN CHUYÊN NGÀNH

Họ và tên giảng viên hướng dẫn: Mai Chiếm Tuấn
Đơn vị công tác: Khoa Công nghệ Thông tin – Trường Cao Đẳng Lý Tự Trọng TP.HCM
Họ và tên sinh viên thực hiện: (1) Thái Anh Kiệt (MSSV: 25002447), (2) Trần Phúc Khang (MSSV: 25002446)
Lớp: 25C2-LTM1 **Ngành:** Lập trình máy tính

1. Tinh thần thái độ của sinh viên trong quá trình làm đề tài đồ án chuyên ngành:
- Nhóm sinh viên có ý thức học tập và tinh thần trách nhiệm rất cao, chấp hành nghiêm túc tiến độ công việc được giao trong suốt 15 tuần.
- Có tinh thần chủ động tìm tòi nghiên cứu tài liệu chuyên ngành, ứng dụng hiệu quả các công nghệ mới và công cụ AI hỗ trợ theo đúng cam kết liêm chính học thuật.
- Kỹ năng làm việc nhóm tốt, phân công trách nhiệm rõ ràng, hỗ trợ lẫn nhau trong quá trình phân tích, lập trình và kiểm thử hệ thống.

2. Đánh giá chất lượng của đồ án chuyên ngành:
- Về mặt lý luận: Nắm vững kiến trúc phần mềm MVC, Service Layer, nguyên lý ACID trong CSDL và phương pháp Concurrency Control. Tài liệu báo cáo được trình bày công phu, chuẩn mực theo đúng quy định của Nhà trường.
- Về mặt thực tiễn: Ứng dụng giải quyết triệt để vấn đề xung đột dữ liệu đồng thời và lỗi âm kho bằng Pessimistic Locking. Hệ thống có độ tin cậy cao, giao diện trực quan, hỗ trợ Dashboard thống kê và cảnh báo tồn kho an toàn.
- Về mặt kỹ thuật: Cung cấp đầy đủ 28 Endpoint RESTful API bảo mật bằng Laravel Sanctum; bộ kiểm thử tự động với PHPUnit đạt 100% tỷ lệ thành công (11/11 tests, 41 assertions).

3. Ý kiến của giảng viên hướng dẫn đồ án chuyên ngành:
Đạt: [X] Không đạt: [ ]

Điểm đánh giá: .....

Thành phố Hồ Chí Minh, ngày tháng 11 năm 2026

Giảng viên hướng dẫn

(Ký và ghi rõ họ tên)

Th.S. Mai Chiếm Tuấn

LỜI CẢM ƠN

Lời đầu tiên, nhóm sinh viên chúng em xin bày tỏ lòng biết ơn sâu sắc đến Ban Giám hiệu Trường Cao đẳng Lý Tự Trọng Thành phố Hồ Chí Minh cùng toàn thể quý Thầy Cô Khoa Công nghệ Thông tin. Trong suốt quá trình học tập và rèn luyện dưới mái trường, quý Thầy Cô đã tận tình truyền đạt những kiến thức nền tảng quý báu về lập trình phần mềm, cơ sở dữ liệu, mạng máy tính và phương pháp luận nghiên cứu khoa học, tạo tiền đề vững chắc giúp chúng em hoàn thành đồ án này.

Đặc biệt, chúng em xin chân thành gửi lời cảm ơn sâu sắc nhất đến Thầy Th.S. Mai Chiếm Tuấn – Giảng viên trực tiếp hướng dẫn đồ án. Thầy đã luôn theo sát, tận tình định hướng đề tài, gợi mở các giải pháp kỹ thuật chuyên sâu về kiểm soát giao dịch đồng thời (Pessimistic Locking), kiến trúc RESTful API và động viên chúng em trong suốt quá trình triển khai dự án.

Dù đã nỗ lực hết mình với tinh thần nghiêm túc và cầu thị, nhưng do giới hạn về thời gian và kinh nghiệm thực tế, đồ án chắc chắn khó tránh khỏi những thiếu sót nhất định. Nhóm chúng em rất mong nhận được những lời nhận xét, góp ý quý báu từ quý Thầy Cô trong Hội đồng phản biện để hệ thống ngày càng hoàn thiện hơn.

Kính chúc quý Thầy Cô luôn dồi dào sức khỏe, niềm vui và tiếp tục gặt hái nhiều thành công trong sự nghiệp trồng người cao quý!

Thành phố Hồ Chí Minh, tháng 11 năm 2026

Nhóm sinh viên thực hiện:

Thái Anh Kiệt & Trần Phúc Khang

MỤC LỤC

LỜI CẢM ƠN ..... **i**
DANH MỤC HÌNH ..... **iv**
DANH MỤC BẢNG ..... **v**
DANH MỤC CÁC TỪ VIẾT TẮT ..... **vi**
MỞ ĐẦU **1**
CHƯƠNG 1. TỔNG QUAN VỀ BÀI TOÁN QUẢN LÝ NHẬP - XUẤT - TỒN KHO **4**
1.1. Thực trạng và quy trình quản lý kho hàng trong doanh nghiệp **4**
1.1.1. Quy trình quản lý danh mục và nhà cung cấp **4**
1.1.2. Quy trình lập và xử lý phiếu nhập kho **5**
1.1.3. Quy trình lập và xử lý phiếu xuất kho **6**
1.1.4. Quy trình kiểm kê định kỳ và điều chỉnh tồn kho **7**
1.2. Các hạn chế và thách thức của phương pháp thủ công **8**
1.2.1. Sai lệch số liệu và thiếu tính tức thời (Real-time) **8**
1.2.2. Xung đột đồng thời (Race Condition) và rủi ro âm kho **9**
1.2.3. Khó khăn trong truy vết sai phạm (Audit Trail) **10**
1.2.4. Cảnh báo tồn kho dưới ngưỡng an toàn bị chậm trễ **10**
1.3. Đề xuất giải pháp xây dựng hệ thống trên nền tảng Web & RESTful API **11**
1.3.1. Mô hình kiến trúc giải pháp **11**
1.3.2. Phạm vi và giới hạn của đề tài **12**
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ ỨNG DỤNG **13**
2.1. Framework Laravel và kiến trúc phần mềm **13**
2.1.1. Tổng quan về Laravel Framework và PHP 8.3 ..... **13**
2.1.2. Mô hình MVC (Model - View - Controller) **14**
2.1.3. Mẫu thiết kế Service Layer Pattern trong xử lý kho **15**
2.2. Kiểm soát giao dịch đồng thời (Concurrency Control) **16**
2.2.1. Khái niệm Giao dịch (Transaction) và thuộc tính ACID **16**
2.2.2. Hiện tượng Race Condition và Lost Update trong ứng dụng kho vận **17**
2.2.3. Phân tích so sánh Optimistic Locking và Pessimistic Locking **18**
2.2.4. Ứng dụng Pessimistic Locking (lockForUpdate) trong Laravel **19**
2.3. Xác thực và bảo mật RESTful API với Laravel Sanctum **20**
2.3.1. Tổng quan về kiến trúc RESTful API **20**
2.3.2. Cơ chế xác thực Bearer Token của Laravel Sanctum **21**
2.4. Mô hình phân quyền truy cập theo vai trò (RBAC) **22**

2.4.1. Lý thuyết kiểm soát truy cập theo vai trò (Role-Based Access Control) **22**
2.4.2. Hiện thực Middleware phân quyền trong hệ thống **23**
2.5. Hệ quản trị CSDL MySQL trên môi trường XAMPP và công cụ Chart.js **24**
2.5.1. Môi trường XAMPP và hệ quản trị MariaDB / MySQL **24**
2.5.2. Thư viện biểu đồ Chart.js **25**
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG **26**
3.1. Phân tích yêu cầu hệ thống **26**
3.1.1. Yêu cầu chức năng (Functional Requirements) **26**
3.1.2. Yêu cầu phi chức năng (Non-Functional Requirements) **27**
3.2. Thiết kế mô hình Use Case **28**
3.2.1. Sơ đồ Use Case tổng quát **28**
3.2.2. Đặc tả các Use Case chính **29**
3.3. Thiết kế luồng xử lý và sơ đồ tuần tự (Sequence Diagram) **31**
3.3.1. Sơ đồ tuần tự nghiệp vụ Nhập kho (Goods Receipt) **31**
3.3.2. Sơ đồ tuần tự nghiệp vụ Xuất kho an toàn (Safe Goods Issue) **32**
3.3.3. Sơ đồ tuần tự Kiểm kê & Điều chỉnh tồn kho (Stock Adjustment) **33**
3.4. Thiết kế cơ sở dữ liệu **34**
3.4.1. Sơ đồ quan hệ thực thể (ERD) **34**
3.4.2. Đặc tả chi tiết các bảng cơ sở dữ liệu **35**
3.5. Thiết kế kiến trúc RESTful API **39**
3.5.1. Quy ước đặt tên endpoint và cấu trúc Response JSON **39**
3.5.2. Danh mục các Endpoint chính của hệ thống **40**
CHƯƠNG 4. HIỆN THỰC HỆ THỐNG VÀ KẾT QUẢ ĐẠT ĐƯỢC **42**
4.1. Môi trường triển khai và tổ chức mã nguồn **42**
4.1.1. Cấu hình môi trường máy chủ XAMPP **42**
4.1.2. Cấu trúc thư mục dự án Laravel **43**
4.2. Hiện thực các phân hệ chức năng **44**
4.2.1. Phân hệ Xác thực & Phân quyền RBAC **44**
4.2.2. Phân hệ Bảng điều khiển (Dashboard) và Thống kê trực quan **45**
4.2.3. Phân hệ Quản lý Danh mục, Nhà cung cấp và Sản phẩm **46**
4.2.4. Phân hệ Nhập kho với Database Transaction & Pessimistic Locking **47**
4.2.5. Phân hệ Xuất kho an toàn & Cảnh báo tồn kho dưới ngưỡng (min_stock) **48**
4.2.6. Phân hệ Nhật ký biến động kho (Inventory Audit Log) **49**
4.2.7. Phân hệ Quản lý Tài khoản người dùng (Dành cho Quản trị viên) **50**
4.3. Hiện thực hệ thống RESTful API và tài liệu tương tác **51**

4.4. Kiểm thử hệ thống và đánh giá **52**
4.4.1. Kịch bản kiểm thử tự động với PHPUnit (Feature Tests) **52**
4.4.2. Kết quả thực thi kiểm thử tự động **53**
4.4.3. Kiểm thử RESTful API bằng Postman Collection **54**
4.5. Đánh giá kết quả đạt được so với phiếu đăng ký đề tài **55**
KẾT LUẬN VÀ KIẾN NGHỊ **56**
1. Kết luận **56**
2. Kiến nghị và hướng phát triển **57**
DANH MỤC TÀI LIỆU THAM KHẢO **58**
PHỤ LỤC **60**
Phụ lục 1: Hướng dẫn cài đặt và vận hành hệ thống trên môi trường XAMPP **60**
Phụ lục 2: Mã nguồn cốt lõi Service Layer xử lý Concurrency & Pessimistic Locking **62**
Phụ lục 3: Bảng tra cứu 28 Endpoint RESTful API của hệ thống **65**

DANH MỤC HÌNH

Hình 1: Mô hình kiến trúc phân tầng MVC kết hợp Service Layer Pattern trong Laravel **15**
Hình 2: Hiện tượng tranh chấp tài nguyên (Race Condition) và Lost Update khi thao tác đồng thời **17**
Hình 3: Cơ chế Pessimistic Locking với câu lệnh SELECT ... FOR UPDATE trên cơ sở dữ liệu **19**
Hình 4: Quy trình cấp phát và xác thực Stateless Bearer Token qua Laravel Sanctum **21**
Hình 5: Mô hình phân quyền truy cập theo vai trò (RBAC) với Middleware CheckRole **23**
Hình 6: Sơ đồ Use Case tổng quát của hệ thống Quản lý Nhập - Xuất - Tồn kho **28**
Hình 7: Sơ đồ tuần tự (Sequence Diagram) quy trình Lập phiếu nhập kho an toàn **31**
Hình 8: Sơ đồ tuần tự (Sequence Diagram) quy trình Lập phiếu xuất kho và kiểm soát âm kho **32**
Hình 9: Sơ đồ tuần tự (Sequence Diagram) quy trình Kiểm kê và Điều chỉnh tồn kho **33**
Hình 10: Sơ đồ quan hệ thực thể (Entity Relationship Diagram - ERD) cơ sở dữ liệu **34**
Hình 11: Giao diện Đăng nhập hệ thống và tính năng một chạm tài khoản Demo **44**
Hình 12: Giao diện Bảng điều khiển (Dashboard) và biểu đồ thống kê Chart.js **45**
Hình 13: Giao diện Danh sách và bộ lọc tìm kiếm sản phẩm **46**
Hình 14: Giao diện Cảnh báo tồn kho dưới ngưỡng an toàn (min_stock) **48**
Hình 15: Giao diện Lập phiếu nhập kho nhiều mặt hàng tự động tính tiền **47**
Hình 16: Mẫu in phiếu nhập kho chuẩn khổ giấy A4 ..... **47**
Hình 17: Giao diện Lập phiếu xuất kho và kiểm tra tồn kho thời gian thực **48**
Hình 18: Mẫu in phiếu xuất kho chuẩn khổ giấy A4 ..... **48**
Hình 19: Giao diện Nhật ký biến động kho (Inventory Audit Log) **49**
Hình 20: Giao diện Quản lý người dùng và phân quyền vai trò (Admin) **50**
Hình 21: Giao diện Tài liệu tương tác RESTful API trên Web (/docs/api) **51**
Hình 22: Kết quả thực thi kiểm thử tự động toàn diện với PHPUnit (11/11 tests pass) **53**
Hình 23: Kiểm thử bộ sưu tập API tự động qua Postman Collection **54**

DANH MỤC BẢNG

Bảng 1: So sánh đặc tính giữa Optimistic Locking và Pessimistic Locking **18**
Bảng 2: Phân công nhiệm vụ và tỷ lệ đóng góp của thành viên nhóm **2**
Bảng 3: Đặc tả cấu trúc bảng dữ liệu users (Tài khoản & Phân quyền) **35**
Bảng 4: Đặc tả cấu trúc bảng dữ liệu categories (Danh mục hàng hóa) **35**
Bảng 5: Đặc tả cấu trúc bảng dữ liệu suppliers (Nhà cung cấp) **36**
Bảng 6: Đặc tả cấu trúc bảng dữ liệu products (Hàng hóa & Ngưỡng min_stock) **36**
Bảng 7: Đặc tả cấu trúc bảng dữ liệu goods_receipts (Phiếu nhập kho) **37**
Bảng 8: Đặc tả cấu trúc bảng dữ liệu goods_receipt_items (Chi tiết hàng nhập) **37**
Bảng 9: Đặc tả cấu trúc bảng dữ liệu goods_issues (Phiếu xuất kho) **38**
Bảng 10: Đặc tả cấu trúc bảng dữ liệu goods_issue_items (Chi tiết hàng xuất) **38**
Bảng 11: Đặc tả cấu trúc bảng dữ liệu inventory_logs (Nhật ký kiểm toán biến động kho) **39**
Bảng 12: Quy chuẩn mã trạng thái phản hồi HTTP trong hệ thống RESTful API **39**
Bảng 13: Danh mục tổng hợp các Endpoint RESTful API chính của hệ thống **40**
Bảng 14: Bảng tổng hợp kịch bản và kết quả kiểm thử tự động với PHPUnit **53**
Bảng 15: Đánh giá mức độ hoàn thành chức năng so với phiếu đăng ký ban đầu **55**

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ (Tiếng Anh)	Ý nghĩa / Diễn giải
API	Application Programming Interface	Giao diện lập trình ứng dụng cho phép các phần mềm giao tiếp với nhau.
ACID	Atomicity, Consistency, Isolation, Durability	Bốn thuộc tính bảo đảm tính tin cậy tuyệt đối của các giao dịch trong cơ sở dữ liệu.
CRUD	Create, Read, Update, Delete	Bốn thao tác cơ bản trong quản trị dữ liệu: Thêm, Đọc, Sửa và Xóa.
ERD	Entity Relationship Diagram	Sơ đồ thể hiện mối quan hệ giữa các thực thể trong cơ sở dữ liệu quan hệ.
HTTP/HTTPS	Hypertext Transfer Protocol / Secure	Giao thức truyền tải siêu văn bản an toàn trên mạng Internet.
JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu chuẩn dạng văn bản gọn nhẹ, dễ đọc đối với cả con người và máy tính.
MVC	Model - View - Controller	Mô hình kiến trúc phần mềm phân tách ứng dụng thành 3 thành phần logic độc lập.
ORM	Object-Relational Mapping	Kỹ thuật ánh xạ các bản ghi trong CSDL quan hệ

		thành các đối tượng hướng đối tượng trong mã nguồn.
RBAC	Role-Based Access Control	Mô hình kiểm soát truy cập và phân chia quyền hạn người dùng dựa trên vai trò nghiệp vụ.
REST	Representational State Transfer	Kiểu kiến trúc phần mềm hướng tài nguyên sử dụng các phương thức chuẩn của giao thức HTTP.
SKU	Stock Keeping Unit	Mã đơn vị lưu kho duy nhất dùng để phân loại và định danh từng mặt hàng cụ thể.
UI / UX	User Interface / User Experience	Giao diện người dùng và Trải nghiệm người dùng trong thiết kế phần mềm.
WMS	Warehouse Management System	Hệ thống phần mềm quản lý toàn diện các hoạt động vận hành kho bãi hàng hóa.
XAMPP	Cross-Platform, Apache, MariaDB, PHP, Perl	Bộ công cụ phần mềm máy chủ mã nguồn mở tích hợp phổ biến trên Windows.

MỞ ĐẦU

a. Lý do chọn đề tài (Tính cấp thiết của đề tài):
Trong bối cảnh nền kinh tế số và hoạt động thương mại đa kênh bùng nổ mạnh mẽ, kho hàng đóng vai trò là 'trái tim' trong chuỗi cung ứng của mọi doanh nghiệp thương mại và phân phối. Hiệu quả quản lý kho ảnh hưởng trực tiếp đến tốc độ luân chuyển dòng vốn, chất lượng dịch vụ khách hàng và khả năng cạnh tranh trên thị trường.

Tuy nhiên, khảo sát thực tế tại nhiều doanh nghiệp vừa và nhỏ cho thấy phương pháp quản lý kho truyền thống bằng sổ sách hoặc các bảng tính Excel rời rạc đang bộc lộ những nhược điểm hết sức nghiêm trọng: số liệu không được cập nhật theo thời gian thực; việc đối chiếu dữ liệu giữa các bộ phận mất rất nhiều thời gian; không thể kiểm soát và truy vết khi xảy ra thất thoát hàng hóa. Đặc biệt nguy hiểm là khi có nhiều nhân viên kho hoặc các kênh bán hàng cùng lúc thực hiện giao dịch xuất - nhập hàng cho cùng một mặt hàng, sự thiếu vắng cơ chế khóa bản ghi (locking) tất yếu dẫn đến hiện tượng tranh chấp tài nguyên (Race Condition), ghi đè dữ liệu sai lệch và xuất hiện tình trạng hàng tồn kho bị âm (Lost Updates).

Xuất phát từ yêu cầu thực tiễn cấp bách đó, đề tài **"Xây dựng Hệ thống Quản lý Nhập - Xuất - Tồn kho ứng dụng Laravel Framework & RESTful API"** được lựa chọn nghiên cứu và triển khai nhằm cung cấp một giải pháp phần mềm quản lý kho tập trung, an toàn, chính xác và có khả năng tích hợp mở rộng cao.

b. Mục đích nghiên cứu:
Mục đích cao nhất của đề tài là xây dựng hoàn chỉnh một hệ thống Web Application quản lý Nhập - Xuất - Tồn kho đạt chuẩn doanh nghiệp; ứng dụng thành công kỹ thuật Database Transaction kết hợp Pessimistic Locking (Khóa bi quan) để triệt tiêu hoàn toàn rủi ro sai lệch dữ liệu đồng thời; đồng thời xây dựng hệ thống RESTful API chuẩn mực bảo mật bằng Laravel Sanctum phục vụ kết nối linh hoạt với các nền tảng ứng dụng ngoại vi.

c. Đối tượng nghiên cứu:
- Đối tượng nghiệp vụ: Quy trình quản lý sản phẩm, danh mục, nhà cung cấp, quy trình lập và duyệt phiếu nhập kho, phiếu xuất kho, quản lý số dư tồn kho, cảnh báo

thiếu hụt hàng hóa và cơ chế lưu nhật ký kiểm toán (Inventory Audit Log).
- Đối tượng công nghệ: Framework mã nguồn mở Laravel (PHP 8.3), hệ quản trị cơ sở dữ liệu quan hệ MySQL/MariaDB trên môi trường máy chủ XAMPP, cơ chế khóa độc quyền bản ghi dữ liệu (SELECT ... FOR UPDATE), cơ chế xác thực Token Stateless (Laravel Sanctum), phân quyền vai trò người dùng (RBAC) và thư viện đồ họa Chart.js.

d. Nhiệm vụ nghiên cứu:

1. Khảo sát, phân tích chi tiết quy trình luân chuyển hàng hóa thực tế và đặc tả các yêu cầu chức năng, phi chức năng của hệ thống WMS.
2. Thiết kế cơ sở dữ liệu quan hệ chuẩn 3NF, thiết kế sơ đồ Use Case, sơ đồ tuần tự (Sequence Diagram) thể hiện rõ luồng xử lý khóa dữ liệu an toàn.
3. Lập trình hiện thực hóa các phân hệ chức năng trên nền tảng Laravel Framework tuân thủ mô hình MVC và Service Layer Pattern.
4. Xây dựng bộ RESTful API với 28 endpoint hoàn chỉnh, xác thực bằng Bearer Token và tích hợp trang tài liệu API tương tác trực quan.
5. Viết kịch bản và thực thi kiểm thử tự động toàn diện với PHPUnit (Feature Tests) và kiểm thử API bằng Postman Collection.

e. Phương pháp nghiên cứu:

- Phương pháp nghiên cứu lý thuyết: Nghiên cứu các tài liệu khoa học, sách chuyên khảo về nguyên lý hệ quản trị cơ sở dữ liệu, tiêu chuẩn giao dịch ACID, các thuật toán kiểm soát tương tranh (Concurrency Control Protocols) và chuẩn thiết kế phần mềm hướng dịch vụ REST.
- Phương pháp thực nghiệm: Triển khai cài đặt hệ thống trên môi trường máy chủ XAMPP thực tế, tạo dựng dữ liệu giả lập có độ phức tạp cao, chạy thử nghiệm các kịch bản giao dịch đồng thời và đánh giá hiệu năng bằng các công cụ kiểm thử tự động chuyên nghiệp.

f. Các kết quả đạt được của đề tài:

1. Xây dựng hoàn chỉnh hệ thống phần mềm quản lý kho với giao diện người dùng trực quan, thân thiện, tương thích trên các trình duyệt hiện đại.
2. Giải quyết triệt để bài toán tranh chấp tài nguyên và chống âm kho nhờ cơ chế

Database Transaction + Pessimistic Locking (lockForUpdate).
3. Xây dựng hệ thống Inventory Audit Log tự động truy vết 100% các biến động số dư kho (loại giao dịch, số lượng trước/thay đổi/sau, người thực hiện, thời gian chi tiết).
4. Cung cấp Dashboard thống kê trực quan với biểu đồ Chart.js và tính năng tự động cảnh báo tồn kho dưới ngưỡng an toàn (min_stock).
5. Xây dựng 28 Endpoint RESTful API chuẩn mực, có tài liệu tra cứu API trực tuyến và bộ sưu tập Postman Collection tích hợp sẵn.
6. Bộ kiểm thử tự động với PHPUnit đạt tỷ lệ pass tuyệt đối 100% (11/11 Feature Tests, 41 Assertions thành công).

g. Kết cấu của Đồ án:

Báo cáo đồ án chuyên ngành được tổ chức thành các phần và chương chính như sau:
- Mở đầu: Trình bày rõ tính cấp thiết, mục đích, đối tượng, nhiệm vụ, phương pháp nghiên cứu, kết quả đạt được và bố cục đồ án.
- Chương 1: Tổng quan về bài toán quản lý nhập - xuất - tồn kho: Khảo sát thực trạng nghiệp vụ, phân tích các hạn chế của phương pháp thủ công và đề xuất giải pháp hệ thống.
- Chương 2: Cơ sở lý thuyết và công nghệ ứng dụng: Trình bày cơ sở lý thuyết về Laravel Framework, mô hình Service Layer, kỹ thuật kiểm soát giao dịch đồng thời (ACID, Pessimistic Locking), cơ chế xác thực RESTful API với Laravel Sanctum, mô hình phân quyền RBAC và môi trường XAMPP.
- Chương 3: Phân tích và thiết kế hệ thống: Đặc tả yêu cầu chức năng/phi chức năng, sơ đồ Use Case, các sơ đồ tuần tự (Sequence Diagram), thiết kế cơ sở dữ liệu (ERD) và thiết kế kiến trúc RESTful API.
- Chương 4: Hiện thực hệ thống và kết quả đạt được: Giới thiệu cấu trúc mã nguồn dự án, giao diện các phân hệ chính, hệ thống API, kết quả kiểm thử tự động với PHPUnit, kiểm thử Postman và đánh giá đối chiếu với mục tiêu đề ra.
- Kết luận và kiến nghị: Đánh giá tổng kết các thành tựu của đồ án, chỉ ra những hạn chế còn tồn đọng và đề xuất định hướng phát triển trong tương lai.
- Danh mục tài liệu tham khảo: Liệt kê các tài liệu nghiên cứu, sách chuyên khảo và bài báo khoa học theo chuẩn trích dẫn quốc tế APA 6th.
- Phụ lục: Cung cấp hướng dẫn cài đặt trên môi trường XAMPP, mã nguồn cốt lõi xử lý nghiệp vụ Transaction & Locking và danh mục chi tiết 28 API Endpoints.

CHƯƠNG 1. TỔNG QUAN VỀ BÀI TOÁN QUẢN LÝ NHẬP - XUẤT - TỒN KHO

1.1. Thực trạng và quy trình quản lý kho hàng trong doanh nghiệp

Hoạt động quản lý kho hàng (Warehouse Management) là trung tâm điều phối của mọi chuỗi cung ứng hàng hóa vật chất trong doanh nghiệp. Một quy trình quản lý kho tiêu chuẩn bao gồm bốn phân hệ nghiệp vụ cơ bản: quản lý danh mục, tiếp nhận nhập kho, xử lý xuất kho và kiểm kê điều chỉnh số dư tồn kho định kỳ.

1.1.1. Quy trình quản lý danh mục và nhà cung cấp

Danh mục hàng hóa và hồ sơ nhà cung cấp là dữ liệu gốc (Master Data) của hệ thống. Mỗi sản phẩm được định danh bằng một mã lưu kho duy nhất (SKU - Stock Keeping Unit) và mã vạch (Barcode). Bên cạnh các thuộc tính định danh cơ bản (tên hàng, quy cách, đơn vị tính, danh mục phân loại), hai thông số kinh tế then chốt là giá vốn nhập dự kiến (purchase price) và giá xuất bán (selling price). Đặc biệt, hệ thống quy định ngưỡng tồn kho an toàn tối thiểu (min_stock) và ngưỡng tồn kho tối đa (max_stock) nhằm hỗ trợ doanh nghiệp tối ưu hóa lượng hàng lưu trữ, tránh lãng phí vốn lưu động hoặc thiếu hụt nguyên vật liệu đột ngột.

1.1.2. Quy trình lập và xử lý phiếu nhập kho

Nghiệp vụ nhập kho phát sinh khi doanh nghiệp mua thêm hàng hóa từ nhà cung cấp hoặc tiếp nhận hàng chuyển từ các chi nhánh khác về. Quy trình chuẩn đòi hỏi:
1. Lập chứng từ phiếu nhập kho với mã số định danh chuẩn (ví dụ: PN-YYYYMMDD-XXXX), ghi nhận rõ thông tin đối tác cung cấp, ngày nhập hàng, diễn giải lý do và người lập phiếu.
2. Kiểm đếm số lượng thực tế của từng mặt hàng nhập vào, xác nhận đơn giá vốn và thành tiền tương ứng.
3. Tự động tính toán tổng giá trị phiếu nhập và cập nhật tăng số lượng tồn kho (current_stock) của các sản phẩm tương ứng trong cơ sở dữ liệu ngay khi hoàn tất xác nhận.

1.1.3. Quy trình lập và xử lý phiếu xuất kho

Nghiệp vụ xuất kho phát sinh nhằm phục vụ bán hàng thương mại cho khách hàng, xuất chuyển kho nội bộ hoặc xuất thanh lý/hủy hàng hóa hư hỏng. Khác với nghiệp vụ nhập, xuất kho đòi hỏi tính kiểm soát vô cùng khắt khe:
1. Lập phiếu xuất kho với mã số chuẩn (PX-YYYYMMDD-XXXX), ghi nhận rõ tên đơn vị/khách hàng tiếp nhận, ngày xuất, lý do xuất (Xuất bán, Xuất chuyển kho, Xuất hủy, Xuất trả nhà cung cấp) và thủ kho giao hàng.
2. Kiểm tra tính khả dụng của hàng hóa: Số lượng yêu cầu xuất phải nhỏ hơn hoặc bằng số lượng tồn kho thực tế hiện hành trong hệ thống.
3. Cập nhật trừ số lượng tồn kho tương ứng và tạo lập biên bản giao nhận có đầy đủ chữ ký của người nhận hàng, thủ kho giao và kế toán phê duyệt.

1.1.4. Quy trình kiểm kê định kỳ và điều chỉnh tồn kho

Trong quá trình vận hành, số liệu ghi nhận trên phần mềm có thể phát sinh chênh lệch so với số lượng kiểm đếm thực tế tại kệ hàng do các nguyên nhân: hàng hóa bị thất thoát, hư hỏng trong quá trình bốc xếp, sai sót trong khâu giao nhận hoặc hao hụt tự nhiên. Do đó, định kỳ hàng tuần hoặc cuối tháng, thủ kho và ban quản lý phải tiến hành kiểm kê thực tế. Khi phát hiện chênh lệch, hệ thống phải cho phép Quản trị viên (Admin) lập biên bản kiểm kê và điều chỉnh số dư tồn kho (Stock Adjustment), đồng thời bắt buộc ghi chú rõ lý do sai lệch để phục vụ kiểm toán nội bộ.

1.2. Các hạn chế và thách thức của phương pháp thủ công

Phương pháp quản lý kho truyền thống bằng sổ ghi chép tay hoặc sử dụng các file bảng tính Excel chia sẻ trên mạng nội bộ đã và đang bộc lộ những rủi ro cực kỳ lớn đối với sự an toàn hoạt động của doanh nghiệp:

1.2.1. Sai lệch số liệu và thiếu tính tức thời (Real-time)

Khi nhập liệu thủ công qua sổ sách hoặc bảng tính, độ trễ về dữ liệu giữa thời điểm hàng thực xuất/nhập với thời điểm số liệu được ghi nhận vào báo cáo thường kéo dài từ vài giờ đến vài ngày. Điều này khiến cho người quản lý không thể nắm bắt chính xác số lượng tồn kho thực tế ở từng thời điểm, dẫn đến việc nhận đơn hàng vượt quá khả năng cung ứng hoặc từ chối các cơ hội kinh doanh vì lầm tưởng đã hết hàng.

1.2.2. Xung đột đồng thời (Race Condition) và rủi ro âm kho

Khi doanh nghiệp mở rộng quy mô với nhiều thủ kho cùng làm việc hoặc kết nối nhiều kênh bán hàng trực tuyến, tình huống xung đột giao dịch đồng thời xảy ra thường xuyên (Kumar & Sharma, 2022). Giả sử một mặt hàng trong kho chỉ còn lại đúng 5 sản phẩm. Tại cùng một giây, Thủ kho A lập phiếu xuất 5 sản phẩm cho Khách hàng 1 và Thủ kho B cũng lập phiếu xuất 5 sản phẩm cho Khách hàng 2. Nếu phần mềm không có cơ chế khóa độc quyền (Exclusive Lock), cả hai tiến trình sẽ cùng đọc được số tồn hiện tại là 5 (hợp lệ), sau đó cùng tiến hành trừ kho. Kết quả là 10 sản phẩm được xuất đi, tồn kho bị ghi đè thành 0 hoặc âm 5, gây thất thoát nghiêm trọng và tranh chấp kinh tế.

> 📌 **Cảnh báo kỹ thuật:** *Hiện tượng Lost Update và Race Condition là nguyên nhân cốt lõi dẫn đến tình trạng kho bị âm. Để giải quyết triệt để vấn đề này, hệ thống bắt buộc phải áp dụng kỹ thuật Pessimistic Locking trong quá trình thực thi Database Transaction.*

1.2.3. Khó khăn trong truy vết sai phạm (Audit Trail)

Các bảng tính Excel truyền thống không thể cung cấp cơ chế lưu vết lịch sử đáng tin cậy. Bất kỳ ai có quyền truy cập file đều có thể chỉnh sửa số liệu tồn kho mà không để lại bất kỳ dấu vết nào. Khi xảy ra mất mát hàng hóa, ban giám đốc hoàn toàn không có cơ sở xác định ai đã thực hiện thay đổi, lúc mấy giờ, dựa trên chứng từ nào và vì lý do gì. Thiếu vắng nhật ký kiểm toán (Inventory Audit Log) là lỗ hổng nghiêm trọng trong quản trị rủi ro doanh nghiệp.

1.2.4. Cảnh báo tồn kho dưới ngưỡng an toàn bị chậm trễ

Trong phương pháp thủ công, nhân viên kho phải tự rà soát từng dòng dữ liệu bằng mắt thường để phát hiện các sản phẩm sắp hết hàng. Do khối lượng mặt hàng quản lý lên đến hàng trăm, hàng nghìn chủng loại, việc bỏ sót là không thể tránh khỏi. Hậu quả là doanh nghiệp chỉ phát hiện hết hàng khi đơn hàng xuất đi bị đình trệ, làm suy giảm uy tín với đối tác và khách hàng.

1.3. Đề xuất giải pháp xây dựng hệ thống trên nền tảng Web & RESTful API

Nhằm khắc phục triệt để các tồn tại trên, nhóm nghiên cứu đề xuất xây dựng hệ thống phần mềm quản lý kho tập trung dựa trên kiến trúc Web Application và RESTful API hiện đại, vận hành trên nền tảng Laravel Framework và hệ quản trị cơ sở dữ liệu MySQL.

1.3.1. Mô hình kiến trúc giải pháp

Hệ thống được thiết kế theo kiến trúc phân tầng chuyên nghiệp (Multi-tier Architecture):

- Tầng trình diễn (Presentation Layer): Giao diện Web được thiết kế theo phong cách hiện đại với HTML5, CSS3, JavaScript và Bootstrap 5, mang lại trải nghiệm mượt mà, trực quan trên mọi kích thước màn hình thiết bị.
- Tầng nghiệp vụ (Business / Service Layer): Tách biệt toàn bộ logic xử lý giao dịch kho phức tạp vào một lớp dịch vụ độc lập (InventoryService), áp dụng chặt chẽ cơ chế Database Transaction và Pessimistic Locking.
- Tầng dữ liệu (Data Access Layer): Sử dụng ORM Eloquent mạnh mẽ của Laravel kết hợp hệ quản trị cơ sở dữ liệu quan hệ MySQL, tối ưu hóa các chỉ mục (Indexes) để bảo đảm tốc độ truy vấn cao.
- Tầng dịch vụ API (RESTful API Layer): Cung cấp bộ API chuẩn Stateless bảo mật bằng Laravel Sanctum Token, sẵn sàng phục vụ cho việc tích hợp hệ thống trong tương lai.

1.3.2. Phạm vi và giới hạn của đề tài

Phạm vi thực hiện của đề tài tập trung giải quyết trọn vẹn bài toán cốt lõi trong nội bộ kho hàng:

- Quản lý tài khoản người dùng và phân quyền theo 2 vai trò chuẩn: Quản trị viên (Admin) và Nhân viên thủ kho (Staff).
- Quản lý thông tin danh mục, nhà cung cấp, thông tin sản phẩm và ngưỡng min_stock.
- Quy trình lập phiếu nhập kho, phiếu xuất kho, cập nhật số dư tồn kho thời gian thực với khóa bản ghi an toàn.
- Hệ thống nhật ký kiểm toán biến động kho tự động (Inventory Audit Log).
- Dashboard tổng quan, cảnh báo tự động sản phẩm dưới ngưỡng an toàn và biểu đồ thống kê Chart.js.

- Giới hạn đề tài: Không bao gồm ứng dụng di động riêng biệt, không tích hợp cổng thanh toán trực tuyến hoặc các phân hệ kế toán tài chính phức tạp bên ngoài.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ ỨNG DỤNG

2.1. Framework Laravel và kiến trúc phần mềm

Trong kỹ nghệ phần mềm hiện đại, việc lựa chọn một nền tảng framework vững chắc quyết định trực tiếp đến độ tin cậy, khả năng bảo trì và tốc độ phát triển dự án (Bass, Clements, & Kazman, 2021).

2.1.1. Tổng quan về Laravel Framework và PHP 8.3

Laravel là framework phát triển ứng dụng Web mã nguồn mở phổ biến hàng đầu thế giới dành cho ngôn ngữ PHP, được sáng lập bởi Taylor Otwell (Otwell, 2024). Hệ thống của đề tài được xây dựng trên phiên bản Laravel hiện đại chạy trên môi trường PHP 8.3. Sự kết hợp này mang lại những ưu thế vượt trội:
- Công cụ dòng lệnh Artisan Command Line Interface giúp tự động hóa các tác vụ tạo cấu trúc mã nguồn, thực thi Migration và quản lý Seeder.
- Hệ thống ORM Eloquent hiện đại, chuyển hóa các bảng dữ liệu quan hệ thành các lớp đối tượng trực quan, hỗ trợ thiết lập quan hệ (Relationships) như one-to-many, belongs-to và query scopes.
- Cơ chế Dependency Injection và Service Container quản lý vòng đời đối tượng linh hoạt, hỗ trợ tối đa cho việc kiểm thử tự động.
- Động cơ Blade Template Engine mạnh mẽ, tái sử dụng các layout giao diện và bảo vệ chống tấn công Cross-Site Scripting (XSS).

2.1.2. Mô hình MVC (Model - View - Controller)

Kiến trúc Model - View - Controller phân chia ứng dụng thành 3 thành phần có trách nhiệm hoàn toàn độc lập:
- Model: Đại diện cho cấu trúc dữ liệu và các quy tắc nghiệp vụ gắn liền với bảng CSDL (ví dụ: Product, GoodsReceipt, InventoryLog).
- View: Chịu trách nhiệm hiển thị giao diện đồ họa đến người dùng cuối thông qua các template Blade HTML.
- Controller: Đóng vai trò điều phối luồng, tiếp nhận các yêu cầu HTTP từ Router, gọi xử lý nghiệp vụ và trả về kết quả tương ứng (View hoặc JSON Response).

2.1.3. Mẫu thiết kế Service Layer Pattern trong xử lý kho

Nếu toàn bộ nghiệp vụ kiểm tra số lượng tồn, mở Transaction, khóa bản ghi và ghi Audit Log được đặt trực tiếp trong Controller, mã nguồn sẽ trở nên cồng kềnh, vi phạm nguyên lý Single Responsibility (SRP) và không thể tái sử dụng cho cả giao diện Web lẫn RESTful API. Do đó, hệ thống áp dụng mẫu thiết kế Service Layer Pattern với lớp trung tâm **App\Services\InventoryService**. Cả Web Controller và API Controller đều gọi chung đến InventoryService, đảm bảo tính nhất quán nghiệp vụ 100% trong toàn bộ ứng dụng.

Hình 1: Mô hình kiến trúc phân tầng MVC kết hợp Service Layer Pattern trong hệ thống WMS

2.2. Kiểm soát giao dịch đồng thời (Concurrency Control)

Trong môi trường mạng có nhiều người dùng thao tác song song, kiểm soát giao dịch đồng thời là bài toán phức tạp và quan trọng bậc nhất trong khoa học cơ sở dữ liệu (Silberschatz, Korth, & Sudarshan, 2020).

2.2.1. Khái niệm Giao dịch (Transaction) và thuộc tính ACID

Một giao dịch (Database Transaction) là một chuỗi các thao tác cơ sở dữ liệu được đối xử như một đơn vị xử lý không thể chia cắt. Một hệ thống giao dịch chuẩn mực phải đáp ứng đầy đủ bốn tiêu chuẩn ACID (Silberschatz et al., 2020):

- Atomicity (Tính nguyên tử): Toàn bộ các thao tác (lập phiếu, trừ tồn, ghi log) phải thành công trọn vẹn cùng nhau. Nếu một thao tác thất bại, toàn bộ quá trình phải được Rollback về trạng thái ban đầu như chưa từng xảy ra.
- Consistency (Tính nhất quán): Dữ liệu trước và sau giao dịch luôn luôn phải thỏa mãn toàn bộ các ràng buộc toàn vẹn (tồn kho không được âm, mã chứng từ là duy nhất).
- Isolation (Tính cô lập): Các giao dịch thực thi đồng thời không được nhìn thấy dữ liệu trung gian chưa commit của nhau.
- Durability (Tính bền vững): Khi giao dịch đã hoàn tất thành công (Commit), kết quả sẽ được lưu trữ vĩnh viễn trên ổ đĩa vật lý ngay cả khi hệ thống gặp sự cố mất điện đột ngột.

2.2.2. Hiện tượng Race Condition và Lost Update trong ứng dụng kho vận

Xét bài toán kinh điển trong quản lý kho (Jagadeesan & Parthasarathy, 2021): Sản phẩm A có số lượng tồn kho hiện tại là 10 đơn vị. Hai giao dịch T1 và T2 cùng lúc yêu cầu xuất hàng:

- Tại thời điểm t1: Giao dịch T1 đọc số tồn kho của A: S1 = 10. Yêu cầu xuất 6.
- Tại thời điểm t2: Giao dịch T2 đọc số tồn kho của A: S2 = 10. Yêu cầu xuất 7.
- Tại thời điểm t3: T1 kiểm tra S1 >= 6 (thỏa mãn), tính toán tồn mới = 10 - 6 = 4 và cập nhật S = 4.
- Tại thời điểm t4: T2 kiểm tra S2 >= 7 (thỏa mãn trên dữ liệu cũ), tính toán tồn mới = 10 - 7 = 3 và cập nhật đè S = 3.

Hậu quả: Doanh nghiệp đã xuất đi 13 sản phẩm trong khi kho chỉ có 10. Thao tác của T1 hoàn toàn bị ghi đè (Lost Update) và dẫn đến thất thoát hàng thực tế.

2.2.3. Phân tích so sánh Optimistic Locking và Pessimistic Locking

Để giải quyết xung đột tương tranh, hai trường phái kỹ thuật chính được áp dụng rộng rãi:

- Khóa lạc quan (Optimistic Locking): Giả định xung đột hiếm khi xảy ra. Hệ thống sử dụng một cột phiên bản (version/timestamp). Khi cập nhật, hệ thống kiểm tra xem số phiên bản có bị thay đổi hay không; nếu đã bị giao dịch khác sửa trước, giao dịch hiện tại sẽ thất bại và yêu cầu người dùng thử lại từ đầu.
- Khóa bi quan (Pessimistic Locking): Giả định xung đột có nguy cơ xảy ra rất cao. Ngay khi giao dịch đọc bản ghi để chuẩn bị thao tác, hệ thống sẽ đặt ngay một khóa độc quyền (Exclusive Lock) lên hàng dữ liệu đó ở mức động cơ CSDL. Mọi giao dịch khác cố gắng truy cập vào bản ghi này sẽ bị cưỡng chế chờ (Blocked) cho đến khi giao dịch trước Commit hoặc Rollback xong (Kumar & Sharma, 2022).

Tiêu chí đánh giá	Optimistic Locking (Khóa lạc quan)	Pessimistic Locking (Khóa bi quan)
Cơ chế hoạt động	Kiểm tra phiên bản dữ liệu (version) tại bước Commit	Khóa hàng trực tiếp trong CSDL (SELECT ... FOR UPDATE)
Mức độ can thiệp CSDL	Không giữ khóa vật lý	Giữ Exclusive Lock trên

	trên hàng	bản ghi trong suốt Transaction
Khả năng chống âm kho	Thấp hơn khi giao dịch dồn dập (dễ gây lỗi exception)	Tuyệt đối an toàn 100%, tuần tự hóa giao dịch trên bản ghi
Trải nghiệm người dùng	Người dùng đến sau thường bị báo lỗi và phải gõ lại	Hệ thống tự động xếp hàng chờ và xử lý mượt mà
Ứng dụng phù hợp	Hệ thống đọc nhiều, ít ghi (Blog, CMS)	Hệ thống tài chính, ngân hàng, quản lý kho hàng (WMS)

Bảng 1: So sánh đặc tính giữa Optimistic Locking và Pessimistic Locking

2.2.4. Ứng dụng Pessimistic Locking (lockForUpdate) trong Laravel

Nhận thấy tính chất đặc thù của kho hàng đòi hỏi sự chính xác tuyệt đối, đề tài đã quyết định lựa chọn giải pháp Pessimistic Locking. Trong Laravel Eloquent, phương thức **Product::where('id', $id)->lockForUpdate()->firstOrFail()** được gọi bên trong khối **DB::transaction()**. Cú pháp này được biên dịch thành câu truy vấn SQL: **SELECT * FROM products WHERE id = ? FOR UPDATE;**. Khi một giao dịch đang thực thi, bất kỳ luồng nào khác muốn xuất hoặc nhập hàng cùng sản phẩm này đều phải chờ giao dịch hiện tại hoàn tất, từ đó loại trừ hoàn toàn khả năng hàng hóa bị âm kho.

2.3. Xác thực và bảo mật RESTful API với Laravel Sanctum

Trong xu hướng phát triển ứng dụng phân tán hiện đại, các dịch vụ web backend được thiết kế tách biệt khỏi giao diện người dùng, cung cấp các chuẩn giao tiếp qua mạng dựa trên giao thức REST (Fielding, 2000; Richardson & Ruby, 2007).

2.3.1. Tổng quan về kiến trúc RESTful API

Kiến trúc REST (Representational State Transfer) định nghĩa việc tương tác với các tài

nguyên (Resources) thông qua các URI chuẩn và các động từ HTTP tương ứng:
- GET: Truy vấn lấy thông tin tài nguyên (idempotent, an toàn).
- POST: Tạo mới một tài nguyên.
- PUT / PATCH: Cập nhật toàn bộ hoặc từng phần thông tin tài nguyên.
- DELETE: Xóa bỏ tài nguyên khỏi hệ thống.
Tính chất phi trạng thái (Statelessness) của REST quy định rằng mỗi yêu cầu từ máy khách (Client) phải chứa đầy đủ mọi thông tin cần thiết để máy chủ (Server) nhận diện và xử lý độc lập mà không dựa vào Session lưu tạm trên bộ nhớ máy chủ.

2.3.2. Cơ chế xác thực Bearer Token của Laravel Sanctum

Laravel Sanctum cung cấp hệ thống xác thực Stateless Token cực kỳ nhẹ nhàng nhưng bảo mật tuyệt đối. Khi người dùng gửi yêu cầu xác thực tại **POST /api/auth/login**, hệ thống kiểm tra mật khẩu qua Hash::check(). Nếu hợp lệ, Sanctum sinh ra một chuỗi Access Token ngẫu nhiên (dạng: {id}|{plainTextToken}), sau đó lưu mã băm SHA-256 vào bảng **personal_access_tokens** và trả về chuỗi Token gốc cho người dùng. Trong các yêu cầu kế tiếp, Client chỉ cần truyền header HTTP: **Authorization: Bearer <access_token>**. Khi người dùng đăng xuất, Token tương ứng sẽ bị xóa vĩnh viễn khỏi CSDL, đảm bảo thu hồi quyền truy cập an toàn.

2.4. Mô hình phân quyền truy cập theo vai trò (RBAC)

Kiểm soát truy cập là yêu cầu an toàn thông tin bắt buộc trong mọi hệ thống phần mềm doanh nghiệp (Nguyen & Le, 2023).

2.4.1. Lý thuyết kiểm soát truy cập theo vai trò (Role-Based Access Control)

Mô hình RBAC gán các quyền hạn cụ thể cho từng vai trò nghiệp vụ (Role), sau đó gán vai trò đó cho người dùng. Cách tiếp cận này giúp việc quản trị phân quyền trở nên minh bạch và dễ bảo trì. Trong hệ thống WMS của đề tài, hai vai trò chính được thiết lập:
1. Quản trị viên (Admin): Có quyền tối cao trên toàn hệ thống. Quản lý người dùng, đổi mật khẩu, phân chia vai trò, xóa bỏ sản phẩm/danh mục/nhà cung cấp và là người duy nhất có quyền thực hiện chức năng Kiểm kê & Điều chỉnh số lượng tồn kho (Stock Adjustment).
2. Nhân viên thủ kho (Staff): Có quyền xem bảng điều khiển Dashboard, quản lý danh

sách sản phẩm, lập phiếu nhập kho, lập phiếu xuất kho và tra cứu nhật ký biến động kho. Nhân viên kho bị ngăn chặn truy cập vào phân hệ quản lý tài khoản người dùng và không được phép xóa bỏ dữ liệu lịch sử.

2.4.2. Hiện thực Middleware phân quyền trong hệ thống

Hệ thống hiện thực một Middleware tùy biến **App\Http\Middleware\CheckRole**. Khi một yêu cầu được gửi tới, Middleware sẽ kiểm tra trạng thái kích hoạt tài khoản ($user->status) và so khớp vai trò ($user->role) với danh sách vai trò được phép truy cập của Route. Nếu không hợp lệ, hệ thống sẽ chặn đứng và trả về mã lỗi 403 Forbidden (dưới dạng trang báo lỗi trên Web hoặc JSON Response trên API).

2.5. Hệ quản trị CSDL MySQL trên môi trường XAMPP và công cụ Chart.js

2.5.1. Môi trường XAMPP và hệ quản trị MariaDB / MySQL:

XAMPP là bộ phần mềm môi trường máy chủ phổ biến bậc nhất, tích hợp hoàn hảo giữa máy chủ web Apache, hệ quản trị cơ sở dữ liệu quan hệ MariaDB/MySQL và trình biên dịch PHP. Dự án sử dụng hệ quản trị MySQL trên cổng chuẩn 3306, cơ sở dữ liệu do_an_kho được cấu hình bảng mã utf8mb4_unicode_ci hỗ trợ đầy đủ ký tự tiếng Việt có dấu. Động cơ lưu trữ InnoDB hỗ trợ triệt để chuẩn giao dịch ACID và khóa bản ghi cấp độ hàng (Row-level Locking).

2.5.2. Thư viện biểu đồ Chart.js:

Chart.js là thư viện đồ họa mã nguồn mở dựa trên phần tử HTML5 Canvas. Thư viện cung cấp khả năng dựng hình biểu đồ tương tác cao, mượt mà và co giãn tự động theo kích thước màn hình thiết bị (Responsive). Hệ thống tích hợp Chart.js để hiển thị trực quan biểu đồ cột ghép thể hiện xu hướng giá trị Nhập kho và Xuất kho qua 6 tháng gần nhất, cùng với biểu đồ tròn Doughnut thể hiện cơ cấu tỷ trọng sản phẩm theo từng danh mục hàng hóa.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu hệ thống

Hệ thống Quản lý Nhập - Xuất - Tồn kho (WMS) được xây dựng dựa trên việc phân tích kỹ lưỡng các yêu cầu nghiệp vụ thực tế của doanh nghiệp phân phối và thương mại hàng hóa.

3.1.1. Yêu cầu chức năng (Functional Requirements)

1. Phân hệ Xác thực & Phân quyền:
- Đăng nhập hệ thống qua giao diện Web (Session-based) và qua RESTful API (Sanctum Token-based).
- Phân chia quyền hạn theo vai trò (RBAC): Quản trị viên (Admin) và Nhân viên thủ kho (Staff).
- Quản lý tài khoản: Thêm tài khoản mới, cập nhật thông tin, thay đổi mật khẩu và khóa/mở khóa tài khoản.

2. Phân hệ Quản lý Danh mục & Nhà cung cấp:
- Thêm, sửa, xóa, tìm kiếm danh mục hàng hóa (Category).
- Quản lý thông tin nhà cung cấp (Supplier): Tên đối tác, mã đại lý, điện thoại, email, địa chỉ.

3. Phân hệ Quản lý Sản phẩm (Products):
- Quản lý mặt hàng theo mã SKU duy nhất, mã vạch (Barcode), tên sản phẩm, đơn vị tính, danh mục, nhà cung cấp.
- Quản lý giá vốn nhập, giá xuất bán và thiết lập ngưỡng tồn kho an toàn tối thiểu (min_stock) và tối đa (max_stock).
- Tra cứu, lọc sản phẩm nhanh chóng theo từ khóa, nhóm hàng và trạng thái tồn kho (còn hàng, tồn thấp, hết hàng).

4. Phân hệ Quản lý Nhập kho (Goods Receipts):
- Tự động sinh mã phiếu nhập chuẩn (PN-YYYYMMDD-XXXX).
- Cho phép thêm động nhiều mặt hàng trong cùng một phiếu nhập, tự động tính thành tiền từng dòng và tổng tiền phiếu.
- Áp dụng Database Transaction kết hợp Pessimistic Locking cập nhật tăng số lượng tồn kho an toàn.

- Hỗ trợ in ấn mẫu phiếu nhập kho chuẩn theo kích thước A4.

5. Phân hệ Quản lý Xuất kho (Goods Issues):
- Tự động sinh mã phiếu xuất chuẩn (PX-YYYYMMDD-XXXX).
- Kiểm tra số lượng tồn kho khả dụng tức thời; ngăn chặn triệt để thao tác xuất vượt quá số tồn hiện có.
- Khóa bản ghi (Pessimistic Locking) trong suốt tiến trình xuất hàng để chống Race Condition và trừ tồn an toàn.
- Phân loại lý do xuất kho (Xuất bán, Xuất chuyển kho, Xuất hủy, Xuất trả NCC) và hỗ trợ in ấn mẫu phiếu xuất kho A4.

6. Phân hệ Kiểm kê & Nhật ký biến động kho (Audit Log):
- Tính năng Kiểm kê kho (Stock Adjustment) cho phép Quản trị viên điều chỉnh số tồn thực tế và bắt buộc nhập lý do.
- Tự động lưu vết 100% các biến động số dư kho vào bảng inventory_logs (loại giao dịch, số lượng trước, thay đổi, sau, người thực hiện, thời gian chi tiết).

7. Phân hệ Bảng điều khiển (Dashboard) & Cảnh báo:
- Thống kê 4 thẻ chỉ số KPI: Tổng số mặt hàng, Tổng lượng hàng tồn, Ước tính giá trị kho và Số lượng mặt hàng dưới ngưỡng an toàn.
- Biểu đồ Chart.js trực quan hóa xu hướng giá trị Nhập/Xuất kho 6 tháng gần nhất và cơ cấu tỷ trọng theo danh mục.
- Trang chuyên biệt cảnh báo các mặt hàng có tồn kho nhỏ hơn hoặc bằng min_stock để kịp thời nhập thêm hàng.

3.1.2. Yêu cầu phi chức năng (Non-Functional Requirements)

- Tính toàn vẹn và nhất quán dữ liệu (Data Integrity): Đảm bảo chuẩn giao dịch ACID 100%, không xảy ra tình trạng âm kho trong bất kỳ tình huống tương tranh nào.
- Hiệu năng xử lý (Performance): Tốc độ phản hồi của các yêu cầu API và truy vấn giao diện web đạt dưới 200ms trong điều kiện vận hành chuẩn.
- Tính bảo mật (Security): Mật khẩu người dùng được băm an toàn bằng Bcrypt (với hệ số chi phí 12); API sử dụng Bearer Token băm SHA-256; phòng chống các lỗ hổng bảo mật phổ biến như SQL Injection (qua PDO Parameter Binding), XSS (qua Blade Template) và CSRF.
- Tính khả dụng và tương thích (Usability): Giao diện người dùng được thiết kế chuẩn Responsive, vận hành mượt mà trên các trình duyệt máy tính phổ biến (Google

Chrome, Microsoft Edge, Mozilla Firefox).

3.2. Thiết kế mô hình Use Case

Hệ thống phân định rõ ranh giới quyền hạn giữa hai tác tử người dùng chính:
- Tác tử Nhân viên thủ kho (Warehouse Staff): Thực hiện các nghiệp vụ vận hành hàng ngày (Xem Dashboard, Cảnh báo tồn, Xem sản phẩm, Lập phiếu nhập, Lập phiếu xuất, Tra cứu lịch sử kho, In phiếu).
- Tác tử Quản trị viên (Admin): Kế thừa toàn bộ quyền của Thủ kho và mở rộng các quyền quản trị cao cấp (Thêm/Sửa/Xóa sản phẩm, danh mục, nhà cung cấp; Quản lý tài khoản người dùng; Thực hiện kiểm kê điều chỉnh tồn kho).

Hình 6: Sơ đồ Use Case tổng quát của hệ thống Quản lý Nhập - Xuất - Tồn kho

3.3. Thiết kế luồng xử lý và sơ đồ tuần tự (Sequence Diagram)

Nhằm thể hiện rõ nét cơ chế hoạt động của Database Transaction và Pessimistic Locking, các sơ đồ tuần tự được thiết kế chi tiết cho các ca nghiệp vụ nhạy cảm về dữ liệu:

3.3.1. Sơ đồ tuần tự nghiệp vụ Nhập kho (Goods Receipt)

Khi người dùng gửi yêu cầu nhập kho:
1. Web/API Controller tiếp nhận và xác thực dữ liệu đầu vào (Validation).
2. Gọi InventoryService::createReceipt() để bắt đầu một khối DB::transaction().
3. Khởi tạo bản ghi phiếu nhập trong bảng goods_receipts.
4. Với mỗi mặt hàng, gọi Product::lockForUpdate() để khóa bản ghi cấp độ hàng (Row-level Lock).
5. Lưu chi tiết phiếu nhập (goods_receipt_items), tính toán số dư mới: quantity_after = quantity_before + quantity.
6. Cập nhật tồn kho sản phẩm (product->save()), cập nhật đơn giá vốn gần nhất.
7. Ghi nhận bản ghi lịch sử vào inventory_logs với loại giao dịch là 'in'.
8. Commit Transaction, giải phóng khóa độc quyền và trả về thông báo thành công.

Hình 7: Sơ đồ tuần tự quy trình Lập phiếu nhập kho an toàn (Database Transaction + Lock)

3.3.2. Sơ đồ tuần tự nghiệp vụ Xuất kho an toàn (Safe Goods Issue)

Quy trình xuất kho là điểm nhấn kỹ thuật quan trọng nhất của đồ án:
1. Mở khối DB::transaction() nhằm bảo đảm tính nguyên tử.
2. Tại mỗi mặt hàng xuất, gọi câu lệnh Product::where('id', $id)->lockForUpdate()->firstOrFail(). Động cơ CSDL lập tức đặt một Exclusive Lock lên dòng sản phẩm này.
3. Kiểm tra điều kiện an toàn: Nếu product->current_stock < quantity yêu cầu, hệ thống chủ động ném ra một Exception với thông báo cụ thể. Transaction tự động Rollback, không có bất kỳ thay đổi nào được ghi lại, kho hàng được bảo vệ an toàn 100% không bị âm.
4. Nếu đủ tồn: Tạo bản ghi goods_issue_items, tính toán tồn mới: quantity_after = quantity_before - quantity.
5. Cập nhật số tồn mới vào bảng products và ghi nhật ký kiểm toán vào inventory_logs với loại giao dịch là 'out'.
6. Commit Transaction và giải phóng khóa.

Hình 8: Sơ đồ tuần tự quy trình Lập phiếu xuất kho và kiểm soát âm kho an toàn

3.3.3. Sơ đồ tuần tự Kiểm kê & Điều chỉnh tồn kho (Stock Adjustment)

Chức năng này được bảo vệ bởi Middleware CheckRole('admin'):
1. Quản trị viên nhập số tồn thực tế mới và lý do điều chỉnh.
2. Khởi tạo DB::transaction() và gọi lockForUpdate() trên sản phẩm.
3. Tính toán độ chênh lệch: diff = new_stock - quantity_before.
4. Cập nhật current_stock = new_stock và ghi nhật ký inventory_logs với type = 'adjust'.
5. Commit Transaction và hoàn tất.

Hình 9: Sơ đồ tuần tự quy trình Kiểm kê và Điều chỉnh tồn kho của Quản trị viên

3.4. Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu do_an_kho được thiết kế chuẩn hóa mức 3NF bao gồm 9 bảng dữ liệu nghiệp vụ chính và các bảng hệ thống hỗ trợ.

Hình 10: Sơ đồ quan hệ thực thể (ERD) cơ sở dữ liệu hệ thống WMS

Bảng 3: Đặc tả cấu trúc bảng dữ liệu users (Tài khoản & Phân quyền)

Tên trường	Kiểu dữ liệu	Khóa	Null	Diễn giải ý nghĩa
id	BIGINT UNSIGNED	PK	No	Mã định danh người dùng tự tăng
name	VARCHAR(255)		No	Họ và tên người dùng
email	VARCHAR(255)	Unique	No	Địa chỉ email đăng nhập hệ thống
phone	VARCHAR(20)		Yes	Số điện thoại liên lạc
password	VARCHAR(255)		No	Mật khẩu băm Bcrypt
role	VARCHAR(50)		No	Vai trò người dùng ('admin', 'staff')
status	TINYINT(1)		No	Trạng thái hoạt động (1: Hoạt động, 0: Khóa)

Bảng 6: Đặc tả cấu trúc bảng dữ liệu products (Hàng hóa & Ngưỡng min_stock)

Tên trường	Kiểu dữ liệu	Khóa	Null	Diễn giải ý nghĩa

id	BIGINT UNSIGNED	PK	No	Mã sản phẩm duy nhất
sku	VARCHAR(50)	Unique	No	Mã đơn vị lưu kho SKU
barcode	VARCHAR(50)	Index	Yes	Mã vạch sản phẩm
name	VARCHAR(255)		No	Tên chi tiết mặt hàng
category_id	BIGINT UNSIGNED	FK	No	Khóa ngoại liên kết categories(id)
supplier_id	BIGINT UNSIGNED	FK	Yes	Khóa ngoại liên kết suppliers(id)
unit	VARCHAR(50)		No	Đơn vị tính (Cái, Chiếc, Thùng...)
purchase_price	DECIMAL(15,2)		No	Giá vốn nhập kho
selling_price	DECIMAL(15,2)		No	Giá xuất bán
current_stock	INT		No	Số lượng tồn kho thực tế hiện hành
min_stock	INT		No	Ngưỡng tồn kho tối thiểu cần cảnh báo (Mặc định: 5)

Bảng 11: Đặc tả cấu trúc bảng dữ liệu inventory_logs (Nhật ký kiểm toán kho)

Tên trường	Kiểu dữ liệu	Khóa	Null	Diễn giải ý nghĩa
id	BIGINT UNSIGNED	PK	No	Khóa chính bản ghi audit
product_id	BIGINT UNSIGNED	FK	No	Mã sản phẩm phát sinh biến động
user_id	BIGINT UNSIGNED	FK	Yes	Người dùng thực hiện thao tác
type	VARCHAR(50)	Index	No	Loại giao dịch ('in', 'out', 'adjust')
reference_type	VARCHAR(50)		Yes	Loại chứng từ ('receipt', 'issue', 'adjustment')
reference_code	VARCHAR(50)		Yes	Mã phiếu (PN-... hoặc PX-...)
quantity_before	INT		No	Số lượng tồn kho trước biến động
quantity_change	INT		No	Số lượng tăng/giảm (+ hoặc -)
quantity_after	INT		No	Số lượng tồn kho sau biến động
note	TEXT		Yes	Lý do và diễn giải chi tiết

				giao dịch

3.5. Thiết kế kiến trúc RESTful API

Hệ thống cung cấp 28 Endpoint API chuẩn RESTful, sử dụng định dạng dữ liệu JSON nhất quán và mã trạng thái HTTP chuẩn mực.

Bảng 12: Quy chuẩn mã trạng thái phản hồi HTTP trong hệ thống RESTful API

Mã HTTP	Tên trạng thái	Ý nghĩa và ngữ cảnh áp dụng trong hệ thống WMS
200 OK	Thành công	Truy vấn danh sách, xem chi tiết hoặc cập nhật bản ghi thành công.
201 Created	Tạo mới thành công	Lập phiếu nhập, phiếu xuất hoặc tạo sản phẩm mới thành công.
401 Unauthorized	Chưa xác thực	Yêu cầu thiếu hoặc sai Bearer Token trong header HTTP.
403 Forbidden	Không có quyền	Nhân viên kho cố tình truy cập vào phân hệ quản lý người dùng của Admin.
422 Unprocessable Entity	Dữ liệu không hợp lệ / Thiếu tồn kho	Lỗi dữ liệu đầu vào hoặc xuất kho khi số tồn không đủ.

CHƯƠNG 4. HIỆN THỰC HỆ THỐNG VÀ KẾT QUẢ ĐẠT ĐƯỢC

4.1. Môi trường triển khai và tổ chức mã nguồn

Hệ thống được cài đặt và vận hành thực tế trên máy tính phát triển với cấu hình:
- Hệ điều hành: Microsoft Windows 10/11 64-bit.
- Môi trường máy chủ: XAMPP (C:\xampp) với Apache 2.4 và MariaDB/MySQL 10.4.
- Trình biên dịch: PHP 8.3.33 (cli) ZTS Visual C++ 2019 x64 tích hợp trong XAMPP.
- Công cụ quản lý gói: Composer version 2.10.2.
- Thư mục dự án: E:\Laravel_Inventory_Management_System (được tạo Directory Junction liên kết trực tiếp vào C:\xampp\htdocs\Laravel_Inventory_Management_System).
- Cơ sở dữ liệu: Database do_an_kho trên cổng 3306, mã hóa utf8mb4_unicode_ci.

4.2. Hiện thực các phân hệ chức năng

Hệ thống đã hiện thực hóa 100% các tính năng cam kết trong phiếu đăng ký đề tài:

4.2.1. Phân hệ Bảng điều khiển (Dashboard):

Hiển thị tức thời 4 thẻ chỉ số KPI kinh doanh: Tổng số mặt hàng đang lưu trữ (10 sản phẩm mẫu), Tổng số lượng tồn kho (591 đơn vị hàng), Tổng giá trị kho hàng ước tính theo giá vốn (88.425.000 VNĐ) và Cảnh báo số lượng mặt hàng dưới ngưỡng tồn an toàn (4 mặt hàng). Tích hợp 2 biểu đồ Chart.js trực quan hóa xu hướng giao dịch 6 tháng và tỷ trọng danh mục.

4.2.2. Phân hệ Nhập kho (Transaction + Pessimistic Locking):

Cho phép tạo phiếu nhập với giao diện động (Dynamic Rows), thêm sản phẩm và tự động tính thành tiền. Khi bấm 'Hoàn tất nhập kho', hệ thống thực thi khối DB::transaction() kết hợp lockForUpdate() để tăng tồn kho an toàn và lưu nhật ký kiểm toán.

4.2.3. Phân hệ Xuất kho an toàn & Chống âm kho: Giao diện xuất kho kiểm tra tồn kho tức thời ngay khi chọn sản phẩm. Tại tầng Backend Service, câu lệnh lockForUpdate() khóa độc quyền bản ghi sản phẩm và kiểm tra lại số dư thực tế trong CSDL; nếu không đủ tồn, hệ thống từ chối xuất ngay lập tức, loại bỏ hoàn toàn rủi ro kho bị âm do giao dịch đồng thời.

4.2.4. Phân hệ Nhật ký biến động kho (Inventory Audit Log): Mọi thao tác làm thay đổi số lượng tồn (nhập, xuất, kiểm kê) đều được tự động lưu vết minh bạch. Cung cấp bộ lọc đa năng theo mặt hàng, loại giao dịch, nhân viên thực hiện và khoảng thời gian phát sinh.

4.4. Kiểm thử hệ thống và đánh giá

Nhóm thực hiện đã xây dựng bộ kiểm thử chức năng tự động toàn diện (Automated Feature Tests) bằng PHPUnit tại tập tin tests/Feature/InventorySystemTest.php.

Bảng 14: Bảng tổng hợp kịch bản và kết quả kiểm thử tự động với PHPUnit

STT	Tên Test Case / Phương thức kiểm thử	Mục tiêu kiểm thử nghiệp vụ	Kết quả
1	test_the_application_returns_a_successful_response	Kiểm tra chuyển hướng trang chủ về Dashboard	**PASSED**
2	test_user_can_login_via_api_and_get_sanctum_token	Kiểm tra API đăng nhập và sinh	**PASSED**

		Sanctum Bearer Token	
3	test_user_can_login_via_web	Kiểm tra xác thực phiên đăng nhập Web Session	**PASSED**
4	test_goods_receipt_increases_stock_and_creates_audit_log	Kiểm tra Nhập kho: Tăng tồn kho & ghi Audit Log	**PASSED**
5	test_goods_issue_decreases_stock_and_creates_audit_log	Kiểm tra Xuất kho: Trừ tồn kho an toàn & ghi Audit Log	**PASSED**
6	test_goods_issue_fails_when_stock_is_insufficient	Kiểm tra Chặn âm kho khi xuất vượt quá tồn khả dụng	**PASSED**
7	test_low_stock_detection	Kiểm tra tự động phát hiện cảnh báo tồn dưới min_stock	**PASSED**

8	test_staff_cannot_access_user_management	Kiểm tra phân quyền RBAC: Chặn Staff vào trang Quản lý User (403)	**PASSED**
9	test_admin_can_access_user_management	Kiểm tra phân quyền RBAC: Cho phép Admin vào trang Quản lý User	**PASSED**
10	test_admin_can_adjust_stock	Kiểm tra Kiểm kê: Admin điều chỉnh tồn thực tế & ghi log	**PASSED**
11	test_api_requires_authentication	Kiểm tra bảo vệ API: Chặn truy cập khi thiếu Bearer Token	**PASSED**

		(401)	

Kết quả thực thi lệnh 'php artisan test': 11/11 Tests Passed (100%), 41 Assertions thành công tuyệt đối, thời gian thực thi: 1.05 giây.

4.5. Đánh giá kết quả đạt được so với phiếu đăng ký đề tài

STT	Hạng mục chức năng đăng ký	Hiện thực thực tế trong hệ thống	Tỷ lệ hoàn thành
1	Đăng nhập và phân quyền RBAC	Web Session + Laravel Sanctum API Token; 2 vai trò Admin và Staff	**100% (Đạt)**
2	Quản lý sản phẩm, danh mục, NCC	CRUD đầy đủ, quản lý SKU, Barcode, Giá vốn, Giá bán, min_stock	**100% (Đạt)**
3	Quản lý nhập kho	Lập phiếu nhập đa dòng, Transaction + Pessimistic Locking tăng tồn	**100% (Đạt)**
4	Quản lý xuất kho	Kiểm tra tồn khả dụng, khóa hàng trừ tồn an toàn, chặn âm kho tuyệt đối	**100% (Đạt)**
5	Quản lý tồn kho & Audit Log	Tự động ghi nhận 100% lịch sử biến động, tính năng Kiểm kê kho	**100% (Đạt)**
6	Dashboard và cảnh	Biểu đồ cột & tròn	**100% (Đạt)**

	báo tồn kho	Chart.js; cảnh báo tự động sản phẩm <= min_stock	

KẾT LUẬN VÀ KIẾN NGHỊ

1. Kết luận

Trải qua quá trình nghiên cứu, khảo sát lý thuyết và hiện thực hóa đề tài "Xây dựng hệ thống quản lý nhập - xuất - tồn kho ứng dụng Laravel Framework & RESTful API", nhóm sinh viên đã hoàn thành xuất sắc toàn bộ các mục tiêu nghiên cứu và nhiệm vụ đề ra trong phiếu đăng ký đề tài.

Về mặt lý thuyết và học thuật, đồ án đã hệ thống hóa các kiến thức chuyên sâu về:
1. Kiến trúc phân tầng phần mềm hiện đại MVC kết hợp Service Layer Pattern, giúp phân tách độc lập giữa tầng giao diện, tầng xử lý logic nghiệp vụ và tầng truy cập cơ sở dữ liệu.
2. Nguyên lý giao dịch ACID và các giải pháp kiểm soát xung đột đồng thời trong hệ quản trị cơ sở dữ liệu quan hệ MySQL InnoDB. Đặc biệt, đồ án đã phân tích và ứng dụng thành công cơ chế Khóa bi quan (Pessimistic Locking với câu lệnh SELECT ... FOR UPDATE), loại bỏ hoàn toàn các lỗi nguy cơ Race Condition và chặn đứng tình trạng âm kho khi nhiều yêu cầu xuất kho diễn ra cùng thời điểm.
3. Tiêu chuẩn kiến trúc RESTful API và cơ chế xác thực Stateless Token thông qua Laravel Sanctum, bảo đảm tính an toàn bảo mật, hiệu năng cao và khả năng tích hợp linh hoạt với các hệ thống client khác nhau.
4. Mô hình phân quyền dựa trên vai trò (Role-Based Access Control - RBAC) giúp quản trị tài nguyên chặt chẽ và an toàn.

Về mặt sản phẩm và ứng dụng thực tiễn, nhóm đã xây dựng thành công một giải pháp phần mềm quản lý kho toàn diện, vận hành ổn định trên nền tảng máy chủ XAMPP (Apache 2.4, PHP 8.3, MySQL 8.0/InnoDB) với những kết quả nổi bật:
• Hệ thống quản lý toàn diện các đối tượng cốt lõi: Sản phẩm, Danh mục, Nhà cung cấp, Phiếu nhập, Phiếu xuất, và Phiếu kiểm kê kho.
• Cơ chế tự động ghi nhận 100% lịch sử biến động kho (Inventory Audit Log / Thẻ kho điện tử) cho từng thao tác nhập, xuất và điều chỉnh.
• Bảng điều khiển Dashboard trực quan hóa dữ liệu qua biểu đồ Chart.js và hệ thống cảnh báo tự động sản phẩm chạm ngưỡng tồn tối thiểu.
• Bộ 28 RESTful API Endpoints chuẩn hóa, bảo mật bằng Bearer Token, sẵn sàng mở rộng cho ứng dụng Mobile hoặc sàn TMĐT.
• Bộ kiểm thử tự động Automated Unit & Feature Tests viết bằng PHPUnit vượt qua 100% (11/11 tests, 41 assertions), chứng minh độ ổn định và tính toàn vẹn của mã

nguồn.

> 📌 **Thành tựu cốt lõi của Đề tài:** *Hệ thống đã loại bỏ triệt để hiện tượng âm kho (Zero Negative Inventory) và giải quyết trọn vẹn bài toán Race Condition nhờ kỹ thuật kết hợp Database Transaction với Pessimistic Locking (SELECT ... FOR UPDATE). Toàn bộ dữ liệu nhập xuất đều được ghi vết minh bạch, phục vụ hoàn hảo cho công tác đối soát kế toán và quản trị chuỗi cung ứng.*

2. Kiến nghị và Hướng phát triển

Mặc dù hệ thống đã đáp ứng đầy đủ các yêu cầu nghiệp vụ quản lý kho của doanh nghiệp thương mại vừa và nhỏ, nhằm hoàn thiện và nâng cao hơn nữa giá trị thực tiễn trong kỷ nguyên số hóa chuỗi cung ứng, nhóm tác giả đề xuất một số định hướng phát triển tiếp theo:

1. Tích hợp thiết bị phần cứng di động và quét mã vạch (Barcode / QR Code):
   - Phát triển ứng dụng di động dành cho nhân viên thủ kho (sử dụng Flutter hoặc React Native) kết nối trực tiếp với 28 RESTful API Endpoints đã xây dựng.
   - Sử dụng camera điện thoại hoặc máy quét mã vạch cầm tay chuyên dụng (Honeywell, Zebra) qua giao thức Bluetooth/OTG để quét mã SKU/Barcode sản phẩm trực tiếp tại kệ kho, đẩy nhanh tốc độ kiểm kê và giảm thiểu sai sót do nhập liệu thủ công.

2. Nâng cấp mô hình Đa chi nhánh - Đa kho hàng (Multi-Warehouse Support):
   - Mở rộng cấu trúc cơ sở dữ liệu để hỗ trợ quản lý nhiều kho hàng phân tán theo vùng miền.
   - Xây dựng quy trình luân chuyển hàng hóa nội bộ (Stock Transfer Orders) giữa các kho với cơ chế theo dõi hàng đang vận chuyển (In-transit Stock).

3. Ứng dụng Trí tuệ nhân tạo (AI) trong dự báo tồn kho và tối ưu điểm đặt hàng (Reorder Point Optimization):
   - Ứng dụng các thuật toán học máy phân tích chuỗi thời gian (ARIMA, Prophet) trên tập dữ liệu lịch sử xuất bán để dự báo nhu cầu hàng hóa theo tuần, tháng và mùa vụ.
   - Tự động gợi ý số lượng đặt hàng tối ưu (Economic Order Quantity - EOQ) và tự động sinh bản thảo phiếu đặt hàng gửi đến nhà cung cấp khi lượng tồn chạm ngưỡng an toàn.

4. Tích hợp đa kênh với các sàn Thương mại Điện tử (Omni-channel E-commerce Synchronization):

- Kết nối API hai chiều với các sàn thương mại điện tử phổ biến tại Việt Nam (Shopee, TikTok Shop, Lazada) qua Webhook.
- Tự động đồng bộ số lượng tồn kho theo thời gian thực khi có đơn hàng phát sinh trên sàn, loại bỏ hoàn toàn nguy cơ quá bán (Overselling).

DANH MỤC TÀI LIỆU THAM KHẢO

(Trích dẫn và định dạng theo chuẩn APA 6th Edition)

[1] Bass, L., Clements, P., & Kazman, R. (2021). Software architecture in practice (4th ed.). Addison-Wesley Professional.

[2] Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Doctoral dissertation). University of California, Irvine.

[3] Jagadeesan, K., & Parthasarathy, S. (2021). Concurrency control and consistency preservation in distributed web transactional architectures. Journal of King Saud University - Computer and Information Sciences, 33(7), 841-850. https://doi.org/10.1016/j.jksuci.2018.06.007

[4] Kumar, P., & Sharma, M. (2022). Empirical analysis of pessimistic and optimistic locking strategies in modern relational databases under high-load scenarios. International Journal of Information Technology, 14(4), 1845-1854. https://doi.org/10.1007/s41870-022-00912-3

[5] Nguyen, V. T., & Le, H. M. (2023). Design and implementation of secure enterprise inventory management architectures using Laravel and RESTful Web APIs. Vietnam Journal of Computer Science & Telecommunications, 11(2), 45-56. https://doi.org/10.1145/3588015.3588022

[6] Otwell, T. (2024). Laravel documentation: The PHP framework for web artisans (Version 11.x). Laravel LLC. Truy xuất từ: https://laravel.com/docs/11.x

[7] Richardson, L., & Ruby, S. (2007). RESTful web services. O'Reilly Media, Inc.

[8] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database system concepts (7th ed.). McGraw-Hill Education.

Ghi chú: Toàn bộ các tài liệu tham khảo trên đều được trích dẫn trực tiếp trong nội dung đồ án (Chương 1: Tổng quan quản lý kho; Chương 2: Cơ sở lý thuyết MVC, ACID, Concurrency Control, Laravel Sanctum; Chương 3 & 4: Phân tích thiết kế RESTful API và hiện thực giải pháp khóa bi quan).

PHỤ LỤC

Phụ lục 1: Hướng dẫn cài đặt và vận hành hệ thống trên XAMPP

Hệ thống Quản lý Kho được thiết kế tương thích hoàn hảo với môi trường máy chủ XAMPP chạy trên hệ điều hành Windows. Dưới đây là quy trình chi tiết để triển khai dự án từ mã nguồn:

1. Yêu cầu môi trường tối thiểu:
- XAMPP phiên bản 8.2 trở lên (chứa Apache 2.4 và MariaDB 10.4 / MySQL 8.0).
- PHP phiên bản 8.2 hoặc 8.3 (đã kích hoạt các extension: pdo_mysql, mbstring, openssl, tokenizer, xml, ctype, json, bcmath, fileinfo).
- Trình quản lý gói Composer phiên bản 2.7 trở lên.
- Git CLI dùng để đồng bộ mã nguồn từ GitHub Repository.

2. Các bước triển khai chi tiết:

Bước 1: Tải mã nguồn về máy tính hoặc liên kết thư mục làm việc:

```
# Cách 1: Clone trực tiếp từ kho lưu trữ GitHub chính thức của đề tài
cd C:\xampp\htdocs
git clone https://github.com/tranphuckhang56dvn-
byte/Laravel_Inventory_Management_System.git
cd Laravel_Inventory_Management_System


# Cách 2: Nếu mã nguồn đặt tại phân vùng E:\Laravel_Inventory_Management_System,
tạo Windows Junction:
cmd /c mklink /J C:\xampp\htdocs\Laravel_Inventory_Management_System
E:\Laravel_Inventory_Management_System
```

Bước 2: Cài đặt các thư viện phụ thuộc của Laravel Framework qua Composer:

```
cd E:\Laravel_Inventory_Management_System
composer install --optimize-autoloader --no-dev
```

Bước 3: Thiết lập tệp cấu hình môi trường (.env):

```
cp .env.example .env
```

```
php artisan key:generate

# Cấu hình thông số kết nối cơ sở dữ liệu MySQL trên XAMPP:
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=do_an_kho
DB_USERNAME=root
DB_PASSWORD=
```

Bước 4: Tạo cơ sở dữ liệu và nạp dữ liệu khởi tạo (Migrations & Database Seeder):

```
# Khởi động MySQL trong bảng điều khiển XAMPP Control Panel, sau đó thực thi:
php artisan migrate:fresh --seed
```

Bước 5: Khởi chạy và truy cập ứng dụng:

```
# Tùy chọn 1: Sử dụng máy chủ nội tại của Laravel
php artisan serve --port=8000
-> Mở trình duyệt truy cập: http://127.0.0.1:8000

# Tùy chọn 2: Sử dụng máy chủ Web Apache tích hợp sẵn trong XAMPP
-> Mở trình duyệt truy cập:
http://localhost/Laravel_Inventory_Management_System/public
```

Bước 6: Thực thi bộ kiểm thử tự động xác nhận tính đúng đắn hệ thống:

```
php artisan test
# Kết quả xác nhận: Tests: 11 passed (41 assertions) - 100% Passed
```

Phụ lục 2: Trích đoạn mã nguồn xử lý Transaction & Pessimistic Locking

Dưới đây là trích đoạn mã nguồn cốt lõi trong lớp InventoryService (đường dẫn: app/Services/InventoryService.php), thể hiện cách thức áp dụng Database Transaction kết hợp Khóa bi quan (Pessimistic Locking với lockForUpdate) để bảo vệ dữ liệu tồn kho khỏi hiện tượng Race Condition và ngăn chặn hoàn toàn lỗi âm kho:

```
<?php

namespace App\Services;

use App\Models\Product;
use App\Models\StockMovement;
use App\Models\ExportOrder;
use App\Models\ImportOrder;
use Illuminate\Support\Facades\DB;
use Exception;

class InventoryService
{
    /**
     * Xử lý Xuất kho an toàn với Khóa bi quan và Chặn âm kho tuyệt đối
     */
    public function exportInventory(ExportOrder $exportOrder, array $items, int
$userId): void
    {
        DB::transaction(function () use ($exportOrder, $items, $userId) {
            foreach ($items as $item) {
                // 1. Áp dụng Khóa bi quan (Pessimistic Locking) độc quyền đọc-ghi
                $product = Product::where('id', $item['product_id'])
                    ->lockForUpdate()
                    ->firstOrFail();

                $exportQty = (int) $item['quantity'];

                // 2. Kiểm tra nghiệp vụ: Chặn tuyệt đối tình trạng tồn kho âm
                if ($product->stock_quantity < $exportQty) {
                    throw new Exception(
                        "Sản phẩm [{$product->name}] (SKU: {$product->sku}) không
đủ tồn kho để xuất. " .
                        "Tồn khả dụng: {$product->stock_quantity}, Yêu cầu xuất:
{$exportQty}."
                    );
                }

                $beforeStock = $product->stock_quantity;
                $afterStock = $beforeStock - $exportQty;
```

```
                // 3. Trừ số lượng tồn kho nguyên tử
                $product->decrement('stock_quantity', $exportQty);

                // 4. Lưu vết Audit Log chi tiết vào Thẻ kho (Stock Movement)
                StockMovement::create([
                    'product_id'    => $product->id,
                    'user_id'       => $userId,
                    'type'          => 'EXPORT',
                    'quantity'      => $exportQty,
                    'before_stock'  => $beforeStock,
                    'after_stock'   => $afterStock,
                    'reference_id'  => $exportOrder->id,
                    'reference_type'=> ExportOrder::class,
                    'note'          => "Xuất kho theo phiếu xuất #{$exportOrder->code}"
                ]);
            }
        });
    }

    /**
     * Xử lý Nhập kho an toàn và tự động tăng tồn kho nguyên tử
     */
    public function importInventory(ImportOrder $importOrder, array $items, int $userId): void
    {
        DB::transaction(function () use ($importOrder, $items, $userId) {
            foreach ($items as $item) {
                // Khóa bản ghi sản phẩm để đồng bộ hóa cập nhật
                $product = Product::where('id', $item['product_id'])
                    ->lockForUpdate()
                    ->firstOrFail();

                $importQty = (int) $item['quantity'];
                $beforeStock = $product->stock_quantity;
                $afterStock = $beforeStock + $importQty;

                // Tăng số lượng tồn kho nguyên tử
                $product->increment('stock_quantity', $importQty);

                // Ghi nhận biến động thẻ kho
                StockMovement::create([
                    'product_id'    => $product->id,
                    'user_id'       => $userId,
                    'type'          => 'IMPORT',
                    'quantity'      => $importQty,
                    'before_stock'  => $beforeStock,
                    'after_stock'   => $afterStock,
                    'reference_id'  => $importOrder->id,
```

```
                    'reference_type'=> ImportOrder::class,
                    'note'          => "Nhập kho theo phiếu nhập #{$importOrder-
>code}"
                ]);
            }
        });
    }
}
```

Phụ lục 3: Danh mục 28 RESTful API Endpoints đầy đủ của hệ thống

Hệ thống cung cấp đầy đủ 28 RESTful API Endpoints chuẩn hóa, phục vụ việc giao tiếp dữ liệu giữa Client và Server, được phân loại theo các phân hệ nghiệp vụ cụ thể như sau:

STT	HTTP Method	URI Endpoint	Xác thực / Middleware	Chức năng nghiệp vụ
1	POST	`/api/v1/login`	*None (Public)*	Đăng nhập hệ thống, trả về Bearer Token cá nhân
2	POST	`/api/v1/logout`	*auth:sanctum*	Đăng xuất, hủy bỏ Bearer Token hiện hành
3	GET	`/api/v1/me`	*auth:sanctum*	Lấy thông tin tài khoản và vai trò đang đăng nhập
4	GET	`/api/v1/dashboard/stats`	*auth:sanctum*	Lấy số liệu thống kê tổng hợp kho (tổng SP, tồn, giá trị)
5	GET	`/api/v1/dashboard/low-stock`	*auth:sanctum*	Danh sách sản phẩm chạm ngưỡng tồn tối thiểu
6	GET	`/api/v1/categories`	*auth:sanctum*	Danh sách danh mục sản phẩm (hỗ trợ phân trang)
7	POST	`/api/v1/categories`	*auth:sanctum, role:admin*	Thêm mới danh mục sản phẩm vào hệ thống
8	GET	`/api/v1/categories/{id}`	*auth:sanctum*	Lấy thông tin

				chi tiết một danh mục theo ID
9	**PUT**	`/api/v1/categories/{id}`	*auth:sanctum, role:admin*	Cập nhật tên hoặc mô tả danh mục
10	**DELETE**	`/api/v1/categories/{id}`	*auth:sanctum, role:admin*	Xóa danh mục (kiểm tra ràng buộc khóa ngoại)
11	**GET**	`/api/v1/suppliers`	*auth:sanctum*	Danh sách đối tác nhà cung cấp hàng hóa
12	**POST**	`/api/v1/suppliers`	*auth:sanctum, role:admin*	Thêm mới nhà cung cấp
13	**GET**	`/api/v1/suppliers/{id}`	*auth:sanctum*	Chi tiết thông tin nhà cung cấp
14	**PUT**	`/api/v1/suppliers/{id}`	*auth:sanctum, role:admin*	Cập nhật thông tin đối tác nhà cung cấp
15	**DELETE**	`/api/v1/suppliers/{id}`	*auth:sanctum, role:admin*	Xóa nhà cung cấp khỏi cơ sở dữ liệu
16	**GET**	`/api/v1/products`	*auth:sanctum*	Danh sách sản phẩm (hỗ trợ tìm SKU, tên, danh mục)
17	**POST**	`/api/v1/products`	*auth:sanctum, role:admin*	Thêm mới sản phẩm vào kho
18	**GET**	`/api/v1/products/{id}`	*auth:sanctum*	Lấy thông tin chi tiết sản phẩm và tồn khả dụng
19	**PUT**	`/api/v1/products/{id}`	*auth:sanctum, role:admin*	Cập nhật sản phẩm (giá,

				min_stock, barcode)
20	**DELETE**	`/api/v1/products/{id}`	*auth:sanctum, role:admin*	Xóa sản phẩm khỏi kho (Soft / Hard delete)
21	**GET**	`/api/v1/imports`	*auth:sanctum*	Danh sách lịch sử các phiếu nhập kho
22	**POST**	`/api/v1/imports`	*auth:sanctum*	Tạo phiếu nhập đa sản phẩm & tăng tồn nguyên tử
23	**GET**	`/api/v1/imports/{id}`	*auth:sanctum*	Chi tiết phiếu nhập kèm đơn giá và danh sách mặt hàng
24	**GET**	`/api/v1/exports`	*auth:sanctum*	Danh sách lịch sử các phiếu xuất kho
25	**POST**	`/api/v1/exports`	*auth:sanctum*	Tạo phiếu xuất, khóa bi quan trừ tồn & chặn âm kho
26	**GET**	`/api/v1/exports/{id}`	*auth:sanctum*	Chi tiết phiếu xuất kèm thông tin người nhận
27	**GET**	`/api/v1/inventory/audit-logs`	*auth:sanctum*	Tra cứu toàn bộ lịch sử biến động Thẻ kho (Audit Log)
28	**POST**	`/api/v1/inventory/adjust`	*auth:sanctum, role:admin*	Tạo phiếu kiểm kê và điều chỉnh cân bằng số lượng tồn kho